

Sistemas Multi-Agente y Function Calling

Guion del taller · Hackathon FISC AI AGENTS · UTP, salon 3-303

“Un subagente es un agente normal, llamado desde dentro de una herramienta del orquestador.”

Si se llevan una sola frase, es esa. La otra es que lo que garantiza algo es el código, no el prompt.

Reparto del tiempo

5 min antes	Antes de que entre nadie
10 min	Apertura
18 min	Ejercicio 1 — la descripción ES el prompt
14 min	Ejercicio 2 — que hace run_sync por debajo
30 min (EL CENTRAL)	Ejercicio 3 — el bucle multi-agente
18 min	Ejercicio 4 — la salida estructurada
15 min	Ejercicio 5 — en paralelo, y cuando algo falla
12 min	Ejercicio 6 — el esqueleto de TU MVP
3 min	Cierre
= 120 min	sin contar los 5 min de preparación antes de que entre nadie

120 con apertura y cierre dentro. De esos, unos 60 son de charla y unos 46 de teclado; el resto son corridas, esperas y preguntas. El libreto (libreto.pdf) lleva la cuenta exacta de las palabras y la recalcula sola. Si el slot se acorta, el bloque que se comprime es el 6 — se lo llevan para el sábado. El 3 no se toca, y el 4 tampoco: la salida estructurada es lo que más les cambia el MVP por línea escrita.

5 MIN ANTES

Antes de que entre nadie

Objetivo: Que la primera corrida del salon no falle.

HAZ Proyecta una terminal con fuente grande. 16pt minimo, la gente del fondo tambien existe.

HAZ Corre **python verificar.py** en tu maquina delante de ellos. Si tu verificador pasa, el suyo tambien deberia.

HAZ Ten dos terminales abiertas: una en MODO=test y otra en MODO=gemini. Vas a saltar entre las dos todo el taller.

HAZ Escribe en el pizarron las dos lineas del .env. Las van a pedir cuatro veces.

OJO El 80% de los problemas de instalacion son Python <3.10 o pip apuntando a otro interprete. Preguntalo de entrada: **python --version**.

nota Si alguien llega sin llave de Gemini: MODO=test hace todo el taller. No lo dejes bloqueado esperando la llave.

10 MIN

Apertura

Objetivo: Encuadre. Que sepan que se llevan y por que no es un tema de moda.

DI En el taller pasado vieron agentes, tools y memoria. Hoy vamos a lo que sigue: un agente que manda a otros agentes. Y salen de aqui con el esqueleto del MVP que presentan el sabado.

DI La frase que quiero que se lleven, y la vamos a repetir todo el taller: **un subagente es un agente normal, llamado desde dentro de una herramienta de otro agente**. Eso es todo. No hay framework magico.

PREGUNTA ¿Quien ha tenido un agente que se queda dando vueltas y le quema la cuota? — Levanten la mano. (Casi todos.) Eso tambien lo arreglamos hoy.

DI Dos modos: **test** no usa red ni llave, sirve para verificar que tu codigo corre. **gemini** da respuestas de verdad. Armas en test, pruebas en gemini. Esa disciplina es la diferencia entre llegar al sabado con cuota y sin cuota.

OJO Deja claro AHORA que cada quien necesita SU llave. Una llave compartida entre 30 se muere en el ejercicio 1.

18 MIN

Ejercicio 1 — la descripcion ES el prompt

Objetivo: Que entiendan que el modelo no ve el codigo: ve el nombre, el docstring y los tipos.

HAZ Que todos corran el archivo tal cual, sin leerlo. Primero corre, despues lee.

```
python ejercicios/01_herramientas.py
```

- DI** Esa herramienta tiene un docstring que dice "**Busca cosas.**" Miren lo que el agente hace con eso. No da error: hace una llamada equivocada con total confianza. Eso es lo peligroso.
- DI** El modelo NO VE tu código. Ve tres cosas: el nombre de la función, los tipos y el docstring. Esa es toda la interfaz. Tu docstring no es documentación, es prompt.
- HAZ** Ahora que arreglen el docstring. Un docstring útil dice tres cosas: que devuelve y en que formato, que NO acepta, y que significa el resultado vacío.
- nota Las tres son importantes pero la tercera es la que nadie escribe: si 'Sin coincidencias' significa 'no esta' y no 'fallo', dilo, o el modelo reintenta en bucle.*
- HAZ** Tarea 3 es la que importa: escriben una herramienta desde cero. El cuerpo es una línea, el trabajo es el docstring.
- PREGUNTA** ¿Alguien tiene un docstring que quiera leer en voz alta? — Lee dos o tres y compáralos en vivo. Aquí aprenden más del de al lado que de ti.
- OJO** Tarea 5: piden un archivo que no existe. El agente LEE el error y reintenta en vez de morirse. Eso pasa porque las funciones devuelven el error como texto, no lanzan excepción. Es una decisión de diseño, señálala.

14 MIN

Ejercicio 2 — que hace `run_sync` por debajo

Objetivo: Desmitificar el bucle. Que sepan por que el costo crece más rápido que los pasos.

- DI** Este es el ejercicio más corto y el que más les va a servir el sábado cuando algo falle a las dos de la mañana.

```
mensajes = [objetivo]
mientras no termine:
    respuesta = modelo(mensajes, herramientas)
    si respuesta no pide herramientas: listo
    por cada herramienta pedida:
        salida = ejecutar(herramienta)
        mensajes.append(salida)    <-- SIN ESTO no avanza
```

- DI** Dos consecuencias. Una: el modelo **no tiene memoria** entre llamadas. En cada vuelta se le reenvía el historial completo. La lista es la memoria. Dos: por eso el paso 10 carga los 9 anteriores, y el costo crece más rápido que los pasos.
- HAZ** Que corran y miren el historial impreso. Cada `ModelResponse` es una llamada que pagaron.

```
python ejercicios/02_bajo_el_capo.py
```

- HAZ** Tarea 4: que bajen `request_limit` a 2 en `taller/config.py` y corran otra vez. Que LEAN el mensaje de error.
- DI** Ese tope es lo único que separa un bug de una factura. `UsageLimits` no es higiene, con la cuota gratis es supervivencia.
- PREGUNTA** Si su MVP necesita 30 pasos, ¿qué le pasa al historial? ¿Y al costo? — Dejalos contestar. Esa respuesta es la puerta al ejercicio 3; no la contestes tu.

nota No hay solucion en soluciones/ para este: no hay codigo que escribir, es mirar y entender. Si alguien la busca, dile eso.

30 MIN (EL CENTRAL)

Ejercicio 3 — el bucle multi-agente

Objetivo: Que construyan un orquestador con dos subagentes y VEAN el aislamiento de contexto en numeros.

DI Este es el ejercicio del taller. Lo que salga de aqui es el esqueleto de su MVP.

```
@orquestador.tool
async def explorar(ctx, pregunta: str) -> str:
    resultado = await explorador.run(pregunta, usage=ctx.usage)
    return resultado.output
```

DI Eso es todo el patron. Para el orquestador eso es una herramienta cualquiera; no sabe que por dentro corrio otro agente con su propio bucle.

HAZ Que corran el archivo tal cual. Ya funciona con UN subagente. Que miren el reporte del final.

```
python ejercicios/03_delegacion.py
```

DI Cuatro razones para separar un subagente, y solo una es la principal. **Aislamiento de contexto:** el subagente puede quemar 40.000 tokens leyendo y devolver 200 palabras; esos 40.000 nunca entran al historial del padre. Las otras tres: menos herramientas por decision, modelo por tarea, y permisos.

DI La de permisos miren la en el codigo: el explorador **no tiene** escribir_archivo. No le pedimos que no escriba. No tiene como. Un prompt que dice 'no borres nada' es una recomendacion; una funcion que no existe es una garantia.

HAZ TAREA PRINCIPAL, dales 12 minutos: que escriban el segundo subagente, el escritor. Los TODO dicen que tiene que cumplir, no como se escribe. El explorador de arriba es su ejemplo funcionando: copian la forma, no el texto.

nota Hay solucion en soluciones/03_delegacion.py. Si alguien se traba mas de 5 minutos, que la abra y siga con el grupo. No pierdan 20 minutos en un typo.

HAZ Despues cambian el OBJETIVO a algo que necesite los dos: "Averigua que bug hay y escribe un resumen en reporte.md". Corren y abren sandbox/reporte.md.

OJO El bug del sandbox esta REPARTIDO: notas.txt tiene el sintoma (el login falla con mayusculas, ticket 412) y docs/arquitectura.md tiene la causa (el correo se guarda sin normalizar). No se puede contestar leyendo un solo archivo. Eso es lo que justifica al explorador.

HAZ Tarea 5, hazla tu en vivo: quita `usage=ctx.usage` de una delegacion y corre.

PREGUNTA ¿Que le paso al conteo? ¿Por que conviene que el gasto del hijo cuente para el padre? — Respuesta: si no, el limite del padre no esta limitando nada. Tres subagentes sueltos son tres presupuestos sin techo.

HAZ Tarea 6, el traspaso: que le manden al explorador un encargo vago a proposito — **"Dime que hay en el proyecto."** — y abran reporte.md. Nadie fallo, ningun 429, y el

resultado esta mal. Ese es el costo real de delegar: el padre le encarga al hijo algo peor de lo que cree. Cuando su MVP falle el sabado y los dos agentes "funcionen", que revisen el encargo primero.

*nota El reporte del final mide **tokens que quemo el hijo** contra **caracteres que devolvio**. En MODO=test esa relacion sale al reves: TestModel repite la salida de las herramientas en vez de resumir, y el explorador devuelve mas de lo que quemo. El propio programa lo avisa. Si quieres que se VEA, corre este ejercicio en MODO=gemini en tu maquina y proyectalo: es el unico momento del taller donde vale la pena gastar cuota en publico.*

18 MIN

Ejercicio 4 — la salida estructurada

Objetivo: Que el subagente devuelva un objeto y el padre COMPRUEBE en vez de crear.

- DI** Hasta ahora el explorador devolvía prosa, y el orquestador tenía que creersela. "No encuentre la causa" y "la causa es X" son los dos un string.
- HAZ** Proyecta la clase Hallazgo y el output_type del Agent.
- DI** Con output_type, resultado.output ya no es un string: es un Hallazgo validado por Pydantic antes de que tu código lo vea.
- DI** Cuatro cosas cambian. El padre puede COMPROBAR — **if not hallazgo.causa** es una línea. El modelo ya no elige el formato: si no encaja, Pydantic AI lo hace reintentar. Se acaban los parseos. Y pesa menos en el historial del padre.
- DI** Y lo que mas me gusta: el esquema ES la instruccion. Agregas un campo y el modelo lo llena sin que toques el prompt. Es el ejercicio 1 otra vez, un nivel arriba.
- OJO** Insiste en los campos opcionales: **causa** es opcional a proposito. Un campo obligatorio que el modelo no sabe es una alucinacion garantizada. Lo opcional le da permiso de no saber.

HAZ Dales 7 minutos: escriben la comprobacion y agregan un campo.

nota En MODO=test la causa sale NULA, así que la rama de INCOMPLETO se ve sin gastar llave. Es una ventaja del ejercicio, no un defecto: senalalo.

nota Los reintentos cuestan peticiones. Un esquema con 15 campos obligatorios y descripciones vagas reintenta tres veces por llamada. Pide lo que necesitas para decidir, nada mas.

15 MIN

Ejercicio 5 — en paralelo, y cuando algo falla

Objetivo: Que sepan paralelizar lo que no depende, y que un hijo caído no les mate el demo.

- DI** En el ejercicio 3 los subagentes iban en serie porque había dependencia. Cuando no la hay, esperar al primero es tiempo regalado: `asyncio.gather`.
- HAZ** Proyecta el gather. Señala que NO lleva await en cada `.run()`: gather los espera.
- OJO** La trampa que hay que decir en voz alta: **el reloj baja, el gasto NO**. Y peor — dos rafagas simultaneas se acercan al limite de tasa mas rapido que dos espaciadas. Con cuota gratis, paralelizar de dos en dos, no de diez en diez.

- DI** Y en paralelo van los que LEEN. Dos escritores sobre el mismo archivo arrancando a la vez es una carrera.
- DI** Segunda mitad: los subagentes se caen. Si el hijo lanza una excepcion, se muere el padre y se muere la corrida. Y el sabado uno se va a caer.
- DI** La solucion ya la vieron en el ejercicio 1: devolver el error como TEXTO en vez de lanzarlo. Ahi era una funcion; aqui es un subagente entero. Misma idea.
- OJO** AVISALES ANTES de que empiecen: hay que proteger las DOS delegaciones. El orquestador llama a `en_serie` primero, así que si solo protegen `en_paralelo`, la corrida se sigue muriendo y van a jurar que su código no sirve.
- HAZ** Dales 6 minutos: gather, despues descomentan el raise, ven morir la corrida, y la protegen.
- PREGUNTA** Si un subagente falla y el orquestador sigue, ¿quien decide si la respuesta final todavia sirve? — Respuesta: tu, en el prompt del orquestador. Es producto, no código. Si no se lo dices, se lo inventa.

12 MIN

Ejercicio 6 — el esqueleto de TU MVP

Objetivo: Que salgan con una decision de arquitectura tomada, no con código bonito.

- DI** Este archivo es suyo y se lo llevan. No copien el ejemplo de archivos del ejercicio 3: pongan lo que su equipo va a construir de verdad.
- HAZ** Primero EN PAPEL, 5 minutos, sin tocar el teclado. Las cuatro preguntas del docstring de `06_mi_mvp.py`.
- DI** La pregunta que importa de las cuatro es: por cada subagente, ¿que herramientas **NO** tiene? Si no puedes contestar eso, no sabes que estas construyendo.
- DI** Y el filtro: si un subagente no aisla contexto, no acota permisos, no reduce decisiones y no cambia de modelo, **ese subagente no debería existir**. Hazlo una herramienta normal del orquestador. Multi-agente no es gratis: cada salto es latencia y peticiones.
- HAZ** Pasa por las mesas. No revises código: pregunta "que hace tu MVP en una frase" y "cuantos subagentes y por que". Si no contestan en 20 segundos, ahi esta el trabajo.
- OJO** Si te queda corto el tiempo, este es el bloque que se comprime: el archivo se lo llevan y lo siguen el sabado. Lo que NO se comprime es el ejercicio 3.

nota Este archivo se llama `06_mi_mvp.py`. Se renumero cuando entraron los bloques 4 y 5.

3 MIN

Cierre

Objetivo: Dos ideas que se llevan aunque olviden la sintaxis.

DI Primera: **un subagente es una herramienta**, y su descripción es un prompt. Todo lo del ejercicio 1 aplica igual al ejercicio 3. Si el orquestador no usa a tu subagente, casi siempre el problema es el docstring.

DI Segunda: **lo que garantiza algo es el código, no el prompt**. El explorador no escribe porque no tiene la herramienta. Ningún agente sale del sandbox porque hay una función que rechaza la ruta. Cuando le den herramientas de escritura a un agente el sábado, esa es la diferencia que más les va a importar.

HAZ Enseñales `_ruta_segura()` en `taller/herramientas.py`. Son seis líneas.

DI Y una tercera, gratis: pongan los límites ANTES de la primera corrida, no después del susto.

PREGUNTA ¿Que van a construir? — Dos o tres equipos en voz alta, una frase cada uno. Cierra con eso, no con un resumen tuyo.

Trampas conocidas

Adelantate a estas. Cada una te ahorra cinco manos levantadas.

429 / RESOURCE_EXHAUSTED	Limite de tasa de Google, no es su código. Esperar 60 s o pasar a MODO=test. correr() ya lo explica solo; señala que el repo lo dice con todas sus letras.
404 model not found	El ID del modelo cambio. Confirmar en aistudio.google.com y exportar MODELO_GEMINI=google:<id> . Revisa esto la mañana del taller, no la semana antes.
"Puse mi llave y sigue dando respuestas raras"	Esta en MODO=test. El encabezado de cada corrida imprime modo: — ensenales donde mirar. Es la pregunta numero uno del día.
"Copie .env.ejemplo a .env y no pasa nada"	Revisar que el archivo se llame .env exacto y que este en la raíz del repo. Ojo con Windows escondiendo extensiones y dejando .env.txt .
Se acabo la cuota a mitad del taller	MODO=test hace el taller completo. Nadie se queda sin hacer los ejercicios por esto. Dilo en voz alta cuando pase al primero.
Limite de UsageLimits alcanzado	Es la red de seguridad haciendo su trabajo, no un bug. Si el caso es legitimo, se sube en <code>taller/config.py</code> . Buen momento para repetir por que existe.

Chuleta: el bug del sandbox

El bug plantado en `sandbox/`, para que no lo derives en vivo: **notas.txt** dice que el login falla cuando el correo tiene mayusculas y que es el ticket 412. **docs/arquitectura.md** dice que el correo se guarda tal cual lo escribe el usuario, sin normalizar. Juntando los dos: la comparacion contra la tabla *users* es sensible a mayusculas porque nadie normaliza antes de comparar. **pendientes.md** confirma que el ticket 412 sigue abierto.

Material de reserva

Para el equipo que termina temprano, o para la consulta de arquitectura del final.

Para el equipo que va adelantado: `output_type`

Lo mas util de Pydantic AI que el taller no cubre. Un subagente que devuelve **output_type=Hallazgo** (un BaseModel) en vez de 80 palabras de prosa le da al padre datos verificables en vez de ingles que tiene que volver a interpretar. Para un MVP de hackathon es la mejora de mayor impacto por linea escrita.

Para el equipo que va adelantado: subagentes en paralelo

Los dos subagentes del ejercicio 3 corren en serie. Si dos subagentes no dependen uno del otro, **asyncio.gather** los corre a la vez: son diez lineas y es el momento en que multi-agente empieza a pagar en lugar de solo costar un salto mas.

Si alguien pregunta: ¿y si un subagente falla?

La herramienta del orquestador no atrapa nada, asi que una excepcion del hijo mata la corrida del padre. Para el sabado: envuelve el **await subagente.run(...)** en `try/except` y devuelve el error como TEXTO, igual que hacen las herramientas de `taller/herramientas.py`. El orquestador lo lee y decide, en vez de morirse.

Si alguien pregunta: ¿cuando NO usar subagentes?

Cuando una sola herramienta alcanza. Cada delegacion son peticiones extra y latencia extra, y ninguna de las cuatro razones del ejercicio 3 la justifica sola si el trabajo cabe en una funcion. El filtro esta en el ejercicio 4: si no aísla contexto, no acota permisos, no reduce decisiones y no cambia de modelo, es una herramienta, no un agente.